

浮点数与稳定数值计算实验

解释失败发生在哪一步

先修004、007、010。实验仅用CPU、NumPy与Python标准库，输入由data/config.json给出，均为原创无量纲合成数值。任务包括朴素softmax失败、稳定改写、输入转换丢失、消减和中心差分，不训练模型、不使用网络或付费API。

在当前单元目录运行：

```
python experiment.py --output outputs
python test_experiment.py
```

脚本写出results.json、difference_scan.csv和environment.json。Notebook重新启动内核并从头执行，另写notebook_outputs下同名三文件。重跑会覆盖对应输出目录中的这些文件，不修改data配置；需要旧记录可给脚本另一个--output目录。已提交outputs为一次实际参考输出。

A 先查看dtype能表达什么

```
import numpy as np
from experiment import format_info, vector
f32, f64 = format_info('float32'), format_info('float64')
assert f32['eps'] == 2.0**-23
assert f64['eps'] == 2.0**-52
assert f32['spacing_at_2'] == 2*f32['spacing_at_1']
assert f64['smallest_subnormal'] < f64['smallest_normal']
```

检查点A：解释为什么eps不是最小正数，也不是所有数之间固定的绝对间隔。把(100000000,100000001)分别转成float32和float64，先打印存值，再调用任何归一化函数。记录信息丢失发生在转换还是后续运算。

B 用有限输入制造非有限中间值

```
from experiment import naive_softmax, stable_distribution
z = [1000, 1001, 999]
bad = naive_softmax(z, 'float64')
assert not np.isfinite(bad).all()
good = stable_distribution(z, 'float64')
np.testing.assert_allclose(good['probabilities'],
                           [0.2447284711, 0.6652409558, 0.0900305732],
                           atol=1e-10, rtol=0)
assert abs(float(good['probabilities'].sum()) - 1) < 1e-15
```

检查点B: 写出 m 、三个shifted值、三个指数和分母 S 。把 z 改成(-1000,-999,-1001)重跑, 指出朴素公式失败机制怎样从上溢变成全下溢。稳定式得到的权重与第一组相同, 因为两组分数逐项只相差2000。

naive_softmax是故意允许非有限输出的基线; 它不是生产接口。公开JSON把这些失败数字记成null, 并单独标记naive_all_finite=false; 不要把null当成0参与误差计算。

C 权重为零不表示log权重不存在

```
small = stable_distribution([0, -200], 'float32')
assert small['probabilities'][1] == 0
assert small['zero_tail_terms'] == 1
assert np.isfinite(small['log_probabilities']).all()
assert abs(float(small['log_probabilities'][1]) + 200) < 1e-5
wide = stable_distribution([0, -200], 'float64')
assert wide['probabilities'][1] > 0
```

检查点C: 解释直接取 $\log(0)$ 与使用 $(z-m)-\log S$ 的区别。证明 $\max(z) \leq \text{LSE}(z) \leq \max(z) + \log K$, 并核对主例LSE约1001.4076059644444。对极端(-最大有限数,最大有限数)调用稳定接口, 它会在平移差非有限时拒绝; 本实现没有承诺全部有限输入组合都成功。

D 分离输入丢失和计算稳定性

```
rounded = stable_distribution([100000000, 100000001], 'float32')
preserved = stable_distribution([100000000, 100000001], 'float64')
np.testing.assert_array_equal(rounded['probabilities'], [0.5, 0.5])
np.testing.assert_allclose(preserved['probabilities'],
                           [0.2689414214, 0.7310585786], atol=1e-10)
```

检查点D: 两个算法都用了稳定平移, 为何权重不同? 若只有已转成float32的数组, 再把它升为float64, 能否恢复差1? 必须给出存值证据, 而不能只说精度更高。

E 计算消减的绝对和相对误差

```
from experiment import cancellation
result = cancellation(1e-16, 'float64')
assert result['direct'] == 0
assert result['rationalized'] > 0
assert abs(result['rationalized'] - result['reference']) < 1e-30
```

检查点E: 手算共轭有理化过程, 并说明分母在 $x \geq -1$ 时非零。对配置中四个 x 和两种dtype, 分别记录绝对误差和相对误差。相对误差只有参考非零时才用; 图中显示下限 $1e-18$ 只是绘图选择, 真实表没有被修改。参考计算使用实际转换后的 x 与90位Decimal, 不把它称为无限精度真值。

F 中心差分要同时记录采样点

```
from experiment import central_exp
ok = central_exp(0.7, 1e-5, 'float64')
assert ok['absolute_error'] < 1e-9
try:
    central_exp(0.7, 1e-17, 'float64')
except ArithmeticError as error:
    assert 'Unresolvable' in str(error)
else:
    raise AssertionError('Indistinguishable input was accepted')
```

检查点F：读取CSV中两种dtype的全部26行。先分开computed与拒绝，再比较最大/最小误差。对一次有效计算，比较requested_h、stored_h与两侧actual_step，解释差别。参考是在各dtype实际存储的x处用Decimal计算exp，不是强行与十进制精确0.7比较。

在纸上证明 x^3 的中心差分恰好比导数多 h^2 。再用 $Ah^2 + Bu/h$ 模型解释曲线为何先下降后上升；有限测点的走势不是对所有函数的证明。程序会拒绝无效步长、 $2h$ 溢出、不可区分扰动和非有限函数值。

G 建立失败记录并复跑

测试报告应为9组，100组向量乘2种dtype的独立110位Decimal检查、82个导数参考、9个Fraction三次多项式恒等式和23项拒绝测试，另记录6段文档代码执行。输入数组在函数调用后保持不变，重复脚本与Notebook的三份输出应逐字节一致。

提交一份记录：输入及dtype、实际存值、数学定义域、期待结果、实际中间值、误差参考定义、最终状态。至少包含朴素正溢出、全下溢、稳定尾项下溢、输入转换丢失、消减以及不可分辨步长六类例子。每种都写清为什么仅isfinite还不够，或哪一步isfinite确实抓住了失败。

随附Notebook已用新Python进程中的真实IPython InProcessKernel顺序执行；没有验证浏览器Jupyter界面、跨进程socket内核或新装Anaconda。environment.yml提供建议环境。本实验只记录当前CPU数值结果，不测实际训练质量、低精度加速或其他硬件的次正规数行为。